

Blackthorn

Security Review For Dromos

Collaborative Audit Prepared For: **Dromos**

Lead Security Expert(s): **0x52**
bughuntoor
xiaoming90

Date Audited: **June 23 - July 14, 2026**

Introduction

This Security Review constitutes a preliminary report on Dromos, as part of an ongoing engagement. This report is the midway point before heading into Phase 2 and constitutes a reflection of a series of updates/PRs being reviewed - the "audited" commit here is constantly changing and this serves as a technical perspective into the direction of the audit engagement after Phase 1.

Scope

Repository: dromos-labs/metadex

Audited Commit: aelee28cc05a5d8bd35fe3b0c870bc9909f59861

Files:

- V3/src/art/BokkyPooBahsDateTimeLibrary.sol
- V3/src/art/VeArtProxy.sol
- V3/src/bridge/hyperlane/HyperlaneAdapter.sol
- V3/src/bridge/LeafMessageOrchestrator.sol
- V3/src/bridge/MessageOrchestrator.sol
- V3/src/bridge/RootLocalAdapter.sol
- V3/src/bridge/RootMessageOrchestrator.sol
- V3/src/core/VotingEscrow.sol
- V3/src/factories/GaugeFactory.sol
- V3/src/gauges/Gauge.sol
- V3/src/gauges/V2Gauge.sol
- V3/src/handlers/LeafEmissionsHandler.sol
- V3/src/handlers/RootEmissionsHandler.sol
- V3/src/indexation/PoolFactoryIndexation.sol

- V3/src/interfaces/art/IVeArtProxy.sol
- V3/src/interfaces/bridge/hyperlane/IHyperlaneAdapter.sol
- V3/src/interfaces/bridge/ILeafMessageOrchestrator.sol
- V3/src/interfaces/bridge/IMessageAdapter.sol
- V3/src/interfaces/bridge/IMessageOrchestrator.sol
- V3/src/interfaces/bridge/IRootLocalAdapter.sol
- V3/src/interfaces/bridge/IRootMessageOrchestrator.sol
- V3/src/interfaces/core/IVotingEscrow.sol
- V3/src/interfaces/external/ITokenRegistry.sol
- V3/src/interfaces/external/IWETH.sol
- V3/src/interfaces/factories/IGaugeFactory.sol
- V3/src/interfaces/gauges/IGauge.sol
- V3/src/interfaces/gauges/IV2Gauge.sol
- V3/src/interfaces/handlers/IEmissionsHandler.sol
- V3/src/interfaces/handlers/ILeafEmissionsHandler.sol
- V3/src/interfaces/indexation/IPoolFactoryIndexation.sol
- V3/src/interfaces/minter/IDynamicRateModule.sol
- V3/src/interfaces/minter/IMinter.sol
- V3/src/interfaces/minter/IToken.sol
- V3/src/interfaces/pools/IPool.sol
- V3/src/interfaces/rewards/IFeeDistribution.sol
- V3/src/interfaces/rewards/IIncentiveStreaming.sol
- V3/src/interfaces/rewards/IVotingCheckpoints.sol
- V3/src/interfaces/rewards/IVotingRewardsFactory.sol

- V3/src/interfaces/rewards/IVotingRewardsManager.sol
- V3/src/interfaces/.solhint.json
- V3/src/interfaces/splitter/ISplitter.sol
- V3/src/interfaces/tmp/ITempFactoryRegistry.sol
- V3/src/interfaces/tmp/ITempGauge.sol
- V3/src/interfaces/tmp/ITempReward.sol
- V3/src/interfaces/token/IReceiptTokenExtensions.sol
- V3/src/interfaces/token/ITokenExtensions.sol
- V3/src/interfaces/voter/IGasGuard.sol
- V3/src/interfaces/voter/ILeafVoter.sol
- V3/src/interfaces/voter/IVoterCommon.sol
- V3/src/interfaces/voter/IVoter.sol
- V3/src/interfaces/vpm/IVoterPaymentsModule.sol
- V3/src/libraries/BalanceLogicLibrary.sol
- V3/src/libraries/CheckpointLogicLibrary.sol
- V3/src/libraries/DelegationLogicLibrary.sol
- V3/src/libraries/ParamsLib.sol
- V3/src/libraries/ProtocolConstants.sol
- V3/src/libraries/Roles.sol
- V3/src/libraries/SafeCastLibrary.sol
- V3/src/libraries/VelodromeTimeLibrary.sol
- V3/src/minter/Minter.sol
- V3/src/rewards/FeeDistribution.sol
- V3/src/rewards/IncentiveStreaming.sol

- V3/src/rewards/VotingCheckpoints.sol
- V3/src/rewards/VotingRewardsFactory.sol
- V3/src/rewards/VotingRewardsManager.sol
- V3/src/splitter/Splitter.sol
- V3/src/token/ReceiptToken.sol
- V3/src/token/Token.sol
- V3/src/voter/GasGuard.sol
- V3/src/voter/LeafVoter.sol
- V3/src/voter/Voter.sol
- V3/src/vpm/VoterPaymentsModule.sol

Repository: [dromos-labs/metadex-slipstream](https://github.com/dromos-labs/metadex-slipstream)

Audited Commit: [d94d5b76f0018d432001e702b796b6b990037796](https://github.com/dromos-labs/metadex-slipstream/commit/d94d5b76f0018d432001e702b796b6b990037796)

Files:

- contracts/core/CLFactory.sol
- contracts/core/CLPool.sol
- contracts/core/fees/CustomSwapFeeModule.sol
- contracts/core/fees/CustomUnstakedFeeModule.sol
- contracts/core/fees/DynamicSwapFeeModule.sol
- contracts/core/fees/README.md
- contracts/core/indexation/CLFactoryIndexation.sol
- contracts/core/interfaces/callback/ICLFlashCallback.sol
- contracts/core/interfaces/callback/ICLMintCallback.sol
- contracts/core/interfaces/callback/ICLSwapCallback.sol

- `contracts/core/interfaces/fees/ICustomFeeModule.sol`
- `contracts/core/interfaces/fees/IDiscountRegistry.sol`
- `contracts/core/interfaces/fees/IDynamicFeeModule.sol`
- `contracts/core/interfaces/fees/IFeeModule.sol`
- `contracts/core/interfaces/ICLFactory.sol`
- `contracts/core/interfaces/ICLPool.sol`
- `contracts/core/interfaces/IERC20Minimal.sol`
- `contracts/core/interfaces/IFactoryRegistry.sol`
- `contracts/core/interfaces/IMinter.sol`
- `contracts/core/interfaces/indexation/ICLFactoryIndexation.sol`
- `contracts/core/interfaces/IPoolFactory.sol`
- `contracts/core/interfaces/IPool.sol`
- `contracts/core/interfaces/IVoter.sol`
- `contracts/core/interfaces/IVotingEscrow.sol`
- `contracts/core/interfaces/LICENSE`
- `contracts/core/interfaces/pool/ICLPoolActions.sol`
- `contracts/core/interfaces/pool/ICLPoolConstants.sol`
- `contracts/core/interfaces/pool/ICLPoolDerivedState.sol`
- `contracts/core/interfaces/pool/ICLPoolEvents.sol`
- `contracts/core/interfaces/pool/ICLPoolOwnerActions.sol`
- `contracts/core/interfaces/pool/ICLPoolState.sol`
- `contracts/core/libraries/BitMath.sol`
- `contracts/core/libraries/FixedPoint128.sol`
- `contracts/core/libraries/FixedPoint96.sol`

- `contracts/core/libraries/FullMath.sol`
- `contracts/core/libraries/LICENSE`
- `contracts/core/libraries/LICENSE_MIT`
- `contracts/core/libraries/LiquidityMath.sol`
- `contracts/core/libraries/LowGasSafeMath.sol`
- `contracts/core/libraries/Oracle.sol`
- `contracts/core/libraries/Position.sol`
- `contracts/core/libraries/SafeCast.sol`
- `contracts/core/libraries/SqrtPriceMath.sol`
- `contracts/core/libraries/SwapMath.sol`
- `contracts/core/libraries/TickBitmap.sol`
- `contracts/core/libraries/TickMath.sol`
- `contracts/core/libraries/Tick.sol`
- `contracts/core/libraries/TransferHelper.sol`
- `contracts/core/libraries/UnsafeMath.sol`
- `contracts/core/LICENSE`
- `contracts/core/test/BitMathEchidnaTest.sol`
- `contracts/core/test/BitMathTest.sol`
- `contracts/core/test/CLPoolSwapTest.sol`
- `contracts/core/test/CoreTestERC20.sol`
- `contracts/core/test/Create2Address.sol`
- `contracts/core/test/FullMathEchidnaTest.sol`
- `contracts/core/test/FullMathTest.sol`
- `contracts/core/test/LiquidityMathTest.sol`

- `contracts/core/test/LowGasSafeMathEchidnaTest.sol`
- `contracts/core/test/MockTimeCLPool.sol`
- `contracts/core/test/OracleEchidnaTest.sol`
- `contracts/core/test/OracleTest.sol`
- `contracts/core/test/SqrtPriceMathEchidnaTest.sol`
- `contracts/core/test/SqrtPriceMathTest.sol`
- `contracts/core/test/SwapMathEchidnaTest.sol`
- `contracts/core/test/SwapMathTest.sol`
- `contracts/core/test/TestCLCallee.sol`
- `contracts/core/test/TestCLReentrantCallee.sol`
- `contracts/core/test/TestCLRouter.sol`
- `contracts/core/test/TestCLSwapPay.sol`
- `contracts/core/test/TickBitmapEchidnaTest.sol`
- `contracts/core/test/TickBitmapTest.sol`
- `contracts/core/test/TickEchidnaTest.sol`
- `contracts/core/test/TickMathEchidnaTest.sol`
- `contracts/core/test/TickMathTest.sol`
- `contracts/core/test/TickOverflowSafetyEchidnaTest.sol`
- `contracts/core/test/TickTest.sol`
- `contracts/core/test/UnsafeMathEchidnaTest.sol`
- `contracts/gauge/CLGaugeFactory.sol`
- `contracts/gauge/CLGauge.sol`
- `contracts/gauge/Gauge.sol`
- `contracts/gauge/interfaces/ICLGaugeFactory.sol`

- `contracts/gauge/interfaces/ICLGauge.sol`
- `contracts/gauge/interfaces/ICLPool.sol`
- `contracts/gauge/interfaces/IGauge.sol`
- `contracts/gauge/interfaces/INonfungiblePositionManager.sol`
- `contracts/gauge/interfaces/IReward.sol`
- `contracts/libraries/EnumerableSet.sol`
- `contracts/libraries/VelodromeTimeLibrary.sol`
- `contracts/periphery/base/BlockTimestamp.sol`
- `contracts/periphery/base/ERC721Permit.sol`
- `contracts/periphery/base/LiquidityManagement.sol`
- `contracts/periphery/base/Multicall.sol`
- `contracts/periphery/base/PeripheryImmutableState.sol`
- `contracts/periphery/base/PeripheryPayments.sol`
- `contracts/periphery/base/PeripheryPaymentsWithFee.sol`
- `contracts/periphery/base/PeripheryValidation.sol`
- `contracts/periphery/base/SelfPermit.sol`
- `contracts/periphery/examples/PairFlash.sol`
- `contracts/periphery/interfaces/external/IERC1271.sol`
- `contracts/periphery/interfaces/external/IERC20PermitAllowed.sol`
- `contracts/periphery/interfaces/external/IWETH9.sol`
- `contracts/periphery/interfaces/IERC20Metadata.sol`
- `contracts/periphery/interfaces/IERC4906.sol`
- `contracts/periphery/interfaces/IERC721Permit.sol`
- `contracts/periphery/interfaces/ILpMigrator.sol`

- `contracts/periphery/interfaces/IMixedRouteQuoterV1.sol`
- `contracts/periphery/interfaces/IMixedRouteQuoterV2.sol`
- `contracts/periphery/interfaces/IMulticall.sol`
- `contracts/periphery/interfaces/INonfungiblePositionManager.sol`
- `contracts/periphery/interfaces/INonfungibleTokenPositionDescriptor.sol`
- `contracts/periphery/interfaces/IPeripheryImmutableState.sol`
- `contracts/periphery/interfaces/IPeripheryPayments.sol`
- `contracts/periphery/interfaces/IPeripheryPaymentsWithFee.sol`
- `contracts/periphery/interfaces/IQuoter.sol`
- `contracts/periphery/interfaces/IQuoterV2.sol`
- `contracts/periphery/interfaces/ISelfPermit.sol`
- `contracts/periphery/interfaces/ISwapRouter.sol`
- `contracts/periphery/interfaces/ITickLens.sol`
- `contracts/periphery/lens/CLInterfaceMulticall.sol`
- `contracts/periphery/lens/MixedRouteQuoterV1.sol`
- `contracts/periphery/lens/MixedRouteQuoterV2.sol`
- `contracts/periphery/lens/MixedRouteQuoterV3.sol`
- `contracts/periphery/lens/Quoter.sol`
- `contracts/periphery/lens/QuoterV2.sol`
- `contracts/periphery/lens/README.md`
- `contracts/periphery/lens/TickLens.sol`
- `contracts/periphery/libraries/BytesLib.sol`
- `contracts/periphery/libraries/CallbackValidation.sol`
- `contracts/periphery/libraries/ChainId.sol`

- `contracts/periphery/libraries/HexStrings.sol`
- `contracts/periphery/libraries/LiquidityAmounts.sol`
- `contracts/periphery/libraries/NFTDescriptor.sol`
- `contracts/periphery/libraries/NFTSVG.sol`
- `contracts/periphery/libraries/OracleLibrary.sol`
- `contracts/periphery/libraries/Path.sol`
- `contracts/periphery/libraries/PoolAddress.sol`
- `contracts/periphery/libraries/PoolTicksCounter.sol`
- `contracts/periphery/libraries/PositionKey.sol`
- `contracts/periphery/libraries/PositionValue.sol`
- `contracts/periphery/libraries/SqrtPriceMathPartial.sol`
- `contracts/periphery/libraries/TokenRatioSortOrder.sol`
- `contracts/periphery/libraries/TransferHelper.sol`
- `contracts/periphery/LICENSE`
- `contracts/periphery/LpMigrator.sol`
- `contracts/periphery/NonfungiblePositionManager.sol`
- `contracts/periphery/NonfungibleTokenPositionDescriptor.sol`
- `contracts/periphery/SwapRouter.sol`
- `contracts/periphery/test/Base64Test.sol`
- `contracts/periphery/test/LiquidityAmountsTest.sol`
- `contracts/periphery/test/MockObservable.sol`
- `contracts/periphery/test/MockObservations.sol`
- `contracts/periphery/test/MockTimeNonfungiblePositionManager.sol`
- `contracts/periphery/test/MockTimeSwapRouter.sol`

- `contracts/periphery/test/NFTDescriptorTest.sol`
- `contracts/periphery/test/NFTManagerCallee.sol`
- `contracts/periphery/test/NonfungiblePositionManagerPositionsGasTest.sol`
- `contracts/periphery/test/OracleLibraryTest.sol`
- `contracts/periphery/test/PathTest.sol`
- `contracts/periphery/test/PeripheryImmutableStateTest.sol`
- `contracts/periphery/test/PoolTicksCounterTest.sol`
- `contracts/periphery/test/PositionValueTest.sol`
- `contracts/periphery/test/SelfPermitTest.sol`
- `contracts/periphery/test/TestCallbackValidation.sol`
- `contracts/periphery/test/TestERC20Metadata.sol`
- `contracts/periphery/test/TestERC20PermitAllowed.sol`
- `contracts/periphery/test/TestERC20.sol`
- `contracts/periphery/test/TestMulticall.sol`
- `contracts/periphery/test/TestPositionNFTOwner.sol`
- `contracts/periphery/test/TickLensTest.sol`

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or

related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	9	5

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0x52

A stylized, cursive handwritten signature in white ink, featuring a large, sweeping '0' and 'x' followed by '52'.

@bughuntoor

A stylized, cursive handwritten signature in white ink, featuring a large, sweeping 'D' followed by 'eadh' and '10'.

@xiaoming90

A stylized, cursive handwritten signature in white ink, featuring a large, sweeping 'X' followed by 'iaoming' and '90'.

Issue M-1: Flat GasGuard Budgets Can Crystallize Cross-Chain Double Accrual

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/135>

Vulnerability Detail

Root-side GasGuard validates a vote dispatch's destination gas with a flat formula:

```
baseGas + gaugeCount × gaugeGas + calldataByteGas × totalCalldataSize + safetyMargin
```

The above formula is taken from:

<https://github.com/dromos-labs/metadex/blob/50d92e358c59658e683ed5d22c9ef97581199da2/V3/src/voter/GasGuard.sol#L111>

```
File: GasGuard.sol
103:  function _computeRequiredBudget(
104:      uint256 _chainId,
105:      uint256 _gaugeCount,
106:      uint256 _totalCalldataSize
107:  ) internal view returns (uint256 _required) {
108:      ChainGasConfig memory _config = chainGasConfigs[_chainId];
109:      if (_config.baseGas == 0) revert ChainUnconfigured(_chainId);
110:      _required = uint256(_config.baseGas) + _gaugeCount *
    ↪  uint256(_config.gaugeGas) + uint256(_config.calldataByteGas)
111:          * _totalCalldataSize + uint256(_config.safetyMargin);
112:  }
```

However, the issue is that the gas costs are not flat on the leaf chain. The gas cost grows linearly with the number of weekly boundaries since the chain (`_settleIndex`) or each touched gauge (`_walkGauge`) was last settled.

Thus, there might be a case where a vote that passes the root-side budget check can run out of gas on the leaf before `applyChainState` / `applyTokenAllocation` finish. Then, the whole route reverts, and the leaf keeps its stale `chainRate`, `chainWeight`, and stale gauge allocations, but on the root, the new allocation is already committed.

Following is a more detailed scenario:

1. A voter holds an allocation on leaf chain L targeting gauge G. G has been unsettled for many weeks.
2. The voter re-votes on root, moving their weight from (L, G) to a gauge on another chain, and supplies exactly the GasGuard-minimum gas for L's dispatch.
3. Root commits the new allocation and dispatches in the same tx.
4. On L, `route` → `_handleVote` → `applyChainState/applyTokenAllocation` must first walk the multi-week backlog. The delivered gas is exhausted and `route` reverts.
5. The relayer re-quotes from the same GasGuard parameters and keeps under-funding.
6. Meanwhile, the voter earns on the destination chain with the moved weight and keeps earning on L through the un-unwound allocation to G.
7. Every interim settlement on L (keeper `settleGauge`, local votes, other users' vote messages) permanently locks in that double-earning for the elapsed segment.

Recovery requires someone to notice and either hand-size a larger `process gas` and ignore the quote entirely

There are ways to recover (e.g., hand-size a larger `process gas`), but if settlement occurs in the meantime, whatever has already been accrued at the stale rate/weight is crystallized and cannot be reversed.

Impact

A root-committed vote can repeatedly fail on the leaf, leaving stale rates and allocations active while the new destination accrues emissions simultaneously. Interim leaf settlements permanently crystallize this duplicate accrual, causing irreversible reward misallocation.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/b04dd8d9a3a7d79f657ea0b21a563b2bc1216a8d/Voter.sol#L1134>

Tool Used

Manual Review

Recommendation

Account for settlement backlogs when budgeting gas, or bound settlement work so every validated message can complete reliably

Issue M-2: Latency Reductions Permanently Desynchronize Root and Leaf Emission Windows

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/136>

Vulnerability Detail

In the `internal-docs/specs/leaf-voter/rate-activation-lazy-accumulator-td.md`, the one of the external requirements is as follows:

7. Root must dispatch `Vote` messages with `tAct` monotonically non-decreasing per chain across both user and protocol flavors, regardless of any flavor-specific latency. Specifically, root clamps each dispatched `tAct` to `max(latency-based tAct, last_dispatched_tAct_for_this_chain)`. Combined with the orchestrator's freshness gating, this guarantees every `applyChainState` invocation receives a `tAct` strictly greater than the leaf's stored `lastSettlement`, so the orchestrator's stale-nonce skips lose no accrual.

However, it is possible the root does not dispatch the messages with `tAct` in a monotonic manner due to the missing clamp.

Consider the following scenario:

- Base (Root Chain), Optimism (Leaf Chain)
- Global emission rate = 16.5 TOKEN/sec. Optimism holds 10% of the total vote weight, so its chain rate is 1.65 TOKEN/sec
- Latency was initially set to its conservative 20 minutes.

Assume the following timeline:

- 14:00 - Alice (staker) rebalances her allocation between two OP gauges. $tAct_1 = 14:00 + 20 \text{ min} = 14:20$. Root pre-books OP's emission ceiling at 1.65/s through 14:20. Message 41 (`tAct 14:20`, `rate=1.65/s`) is dispatched to OP.

- 14:01 - The Hyperlane relayer's delivery of Message 41 is momentarily unprofitable at the quoted gas payment due to a gas price spike, so it was delayed before retry.
- 14:05 - Governance executes the planned config change: `setLatency(5 minutes)`
- 14:08 - Bob (large staker) shifts weight, and OP total vote weight increases from 10% to 30%, thus increasing the rate from 1.65 to 4.95/s. $t_{Act_2} = 14:08 + 5min = 14:13$. Since Root's Optimism cursor is already at 14:20, it will internally treat 4.95/s as effective from 14:20. However, the Message 42 dispatches ($t_{Act} \ 14:13, \ rate \ 4.95/s$)
- 14:10 - Message 42 delivers normally. The Optimism leaf settles up to 14:13 at 1.65/s, then starts accruing 4.95/s from 14:13.
- 14:35 - The relayer's retry finally lands 41. Since its nonce (41) is below the last applied (42), the gate discards its rate update.

The result is a split between the root book and the remote leaf book.

For 14:13 to 14:20 (420 seconds), the two ledgers closed the same window at different rates:

- Root ceiling (Base): At the rate of 1.65/s, tokens = ~693
- Optimism leaf accrual: At the rate of 4.95/s, tokens = ~2,079

Over [14:13,14:20], the root Voter ceiling is booked at 1.65 TOKEN/sec, while the remote leaf accumulator will accrue at 4.95 TOKEN/sec, producing a 1,386 TOKEN-equivalent accounting divergence.

Impact

Out-of-order messages can cause the leaf to apply a new rate earlier than the root, permanently accounting the same interval at different rates. This creates excess or missing leaf rewards relative to the root redemption ceiling, potentially causing failed redemptions or lost emissions.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/7960f2c89fc37cf5ede5216d8d1d9c7138e46020/Voter.sol#L287>

Tool Used

Manual Review

Recommendation

Enforce monotonic activation timestamps per chain and use the same clamped timestamp for root accounting and dispatched messages.

Issue M-3: Scoped Rate Updates Break Global Emission Conservation

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/137>

Vulnerability Detail

The `_applyAllocation` resolves the new total weight but recomputes only the vote scope and optional `CHAIN0`:

```
// metadex/V3/src/voter/Voter.sol:663-671
uint128 _totalWeight = _weightOf(totalPoint);
uint256 _emissionRate = MINTER.emissionRate();
for (uint256 _i; _i < _scope.length; ++_i) {
    _recomputeChainRate(_scope[_i], _totalWeight, _emissionRate);
}
if (_includesChain0) _recomputeChainRate(CHAIN0, _totalWeight, _emissionRate);
```

Assume that someone carries a vote that touches chain A and increases `totalWeight` through this loop. A is recomputed against the larger denominator, but the quiet chain B is not. Thus, B's stored rate still embeds the old, smaller denominator and is now too high.

No message is sent to B because B is absent from the dispatch scope.

When B is finally touched, both root and leaf settle the elapsed interval before overwriting the rate. The correction is forward-only. There is no historical debt or clawback. The stale interval has already been closed at the wrong rate.

Consider the following example.

For simplicity's sake, permanent weights are used, and there is no decay.

Let chains A and B each start with weight 50, let the total weight be 100, and let the global emission rate be 100 units per second. `CHAIN0` has zero weight.

Both cached rates initially equal 50:

```
A = floor(50 * 100 / 100) = 50
```

```
B = floor(50 * 100 / 100) = 50
sum = 100
```

Now carry an ordinary position with weight 100 into a vote entirely for A. The post-vote state is A=150, B=50, and total weight=200. Because the vote scope contains only A, we see the root recompute A while leaving B unchanged:

If we advance the two cached rates by ten seconds, they accrue 1,250 units even though the global schedule provides only 1,000:

```
A accrual      = 75 * 10 = 750
B stale accrual = 50 * 10 = 500
scheduled total = 100 * 10 = 1,000
excess         = 1,250 - 1,000 = 250
```

Note: This issue breaks the $\sum \text{chainRate}[c] \leq \text{Minter}.\text{emissionRate}()$ invariant.

Impact

When total voting weight changes, untouched chains retain rates calculated using the old denominator. Their excess accrual is preserved in leaf rewards and root redemption ceilings, allowing aggregate TOKEN minting to exceed the global emission schedule.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/7960f2c89fc37cf5ede5216d8d1d9c7138e46020/Voter.sol#L663>

Tool Used

Manual Review

Recommendation

Whenever total weight changes, atomically settle and recompute all chain rates, then dispatch updates to every leaf.

Issue M-4: Sub-Minimum Rewards Can Permanently Lock CL Positions

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/139>

Vulnerability Detail

Multiple documents stated that withdrawal should never be blocked. For example:

- *"Deposits and withdrawals are never blocked by emission state"* - emission-distribution.md
- *"LP deposits, withdrawals, and emission claims must never be blocked by gauge activation state."* - gauge-deactivation-flow.md
- *Withdrawals always allowed – deposits and withdrawals are never blocked, regardless of emission rate or cap.* - gauge.md

However, this requirement does not always hold.

The `LeafVoter.redeem()` function contains the `MIN_REDEEM_AMOUNT` (`1e6`) check at Line 308 below.

```
File: LeafVoter.sol
301:   function redeem(
302:       uint256 _amount,
303:       address _recipient,
304:       uint256 _gasLimit,
305:       address _refundRecipient
306:   ) external payable nonReentrant {
307:       if (chainStatus == ChainStatus.Paused) revert ChainPaused();
308:       if (_amount < MIN_REDEEM_AMOUNT) revert AmountTooLow();
309:       if (_recipient == address(0)) revert ZeroAddress();
```

`MIN_REDEEM_AMOUNT` mirrors `Minter.MIN_MINT_AMOUNT` so that a nonzero team rate cannot produce a zero Splitter share. However, root-chain CL claims can generate positive per-recipient legs below this minimum, and forced withdrawal provides no mechanism to defer or forfeit such dust.

Assume that Alice provides liquidity to a low-TVL CL pool on Base, and the gauge has referral configured (e.g., `referralShare = 10000 = 1%`).

Some time later, the pool falls out of favor. Since allocations are continuous and decay with each staker's sTOKEN schedule, voters drift away and the gauge's effective emission rate tapers over weeks. Near the end, the entire pool earns dust.

Alice still has an NFT staked, holding 5% of in-range liquidity. She last claimed 12 days ago, so her pending reward is $\approx 5e7$ wei.

Governance decided to suspend the gauge by calling `setEmissionCap(gauge, 0)`. As a result, Alice's pending is frozen at $\approx 5e7$ wei.

Assume that Alice wants to withdraw her NFT. So, the following functions will be triggered:

```
CLGauge._withdraw() => CLGauge._claimPositions() => _computePositionEmissions() =>
↳ _buildMintArrays()
```

which produces two legs:

```
referral leg = 5e7 · 10_000 / 1e6 = 5e5 (Not OK - below the 1e6 floor)
LP leg = 5e7 - 5e5 = 4.95e7 (OK)
```

The `RootEmissionsHandler.handleEmissions` is executed and loops over all the legs. It will attempt to call `LeafVoter.redeem` with `5e5` and since `5e5 < 1e6`, it will revert the entire withdrawal transaction.

Note 1: This can happen even without referral configuration, as any staker whose entire frozen pending lands in $(0, 1e6)$ wei bricks on the LP leg alone. Referral just makes this issue easier to trigger, as it widens the "trap windows" by a factor of `1e6/referralShare`.

Note 2: Even if the `MIN_REDEEM_AMOUNT` check is removed from the `LeafVoter.redeem()`. The revert will still occur deeper at `Minter.mint()` with `AmountTooLow` error message.

Impact

Withdrawing a CL NFT automatically claims its pending emissions. If either the LP or referral reward is positive but below the minimum redemption amount, the claim reverts and rolls back the entire withdrawal.

If the gauge cap is zero, the dust cannot grow past the minimum, leaving the NFT locked indefinitely with no user-controlled escape.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/cfc4cf45cd936556d7ad2d64d60b4d8c8958763a/LeafVoter.sol#L315>

Tool Used

Manual Review

Recommendation

Allow withdrawals to complete independently of reward redemption by storing dust for later, or letting users skip or forfeit it when exiting.

Discussion

spacegliderrrr

I will only add that the issue can be amplified if a chain gets paused/ suspended - as then the LPs might not be able to withdraw their funds for a prolonged time.

Issue M-5: Reward Weights Use Message Delivery Instead of Vote Activation

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/140>

Vulnerability Detail

LeafVoter's canonical gauge-allocation bookkeeping and VotingRewardsManager's fee/incentive reward-weight bookkeeping assign different activation times to the same vote.

LeafVoter pre-settles the gauge under its old allocation through t_{Act} and installs the new gauge contribution at $lastSettlement = t_{Act}$. However, the reward forwarding path discards that frontier, so VotingRewardsManager records the corresponding reward weight at the message-delivery `block.timestamp`.

Consider the following scenario:

- Gauge G and its bound `VotingRewardsManager`, both deployed in the leaf deployment on Base (Root Chain)
- Base is the protocol's root chain, so a message from the root `Voter` to this Base leaf stack follows the root-local leaf path. Root-local message delivery is synchronous in the vote transaction. No Hyperlane message delay.
- Let D denote the timestamp at which that message reaches Base's `LeafVoter`

Assume the following:

1. Bob has permanent allocation $B = 100$ on G . Alice has permanent allocation $A = 100$ parked on `CHAIN0`. An active incentive streams $R = 1$ RWD/s.
2. Alice submits an allocation change to the root `Voter` on Base, moving all 100 weight from `CHAIN0` to G . With a latency of 100 seconds, the canonical activation time is $t_{Act} = 200$.
3. Since delivery is synchronous, the same-transaction delivery and reward-checkpoint timestamp is $D = 100$

4. At $D = 100$, Base's `LeafVoter` settles G with Bob's old allocation through $t_{Act} = 200$ and applies Alice's new contribution at that frontier. The `VotingRewardsManager` nevertheless records Alice's new weight at the delivery/checkpoint timestamp 100.
5. During $[100, 200)$, reward accounting therefore sees Alice and Bob with 100 each and distributes the 100 `RWD` streamed in that interval 50/50. Alice receives 50 `RWD` and Bob receives 50 `RWD`.

However, if we use the actual time where Alice's gauge allocation is active, the correct accounting at t_{Act} would give Bob all 100 `RWD` and Alice 0 `RWD`, because Alice's gauge allocation is not active until timestamp 200.

This behavior also occurs in other root and leaf chain setups where Hyperlane messaging is involved, as long as there is a nonzero gap between delivery and t_{Act} .

Impact

Gauge allocations and reward weights take effect at different times. Depending on message timing, voters may start earning too early or too late, permanently redistributing fees and incentives among voters without increasing total rewards.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/cfc4cf45cd936556d7ad2d64d60b4d8c8958763a/LeafVoter.sol#L1086>

Tool Used

Manual Review

Recommendation

Ensure gauge allocations and reward weights use the same scheduled activation time.

Issue M-6: Out-of-Order Vote Messages Over-Credit Gauges

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/141>

Vulnerability Detail

Assume the following setup:

- Global emission rate $R = 4 \text{ TOKEN/s}$ ($\approx 2.42\text{M/week}$)
- Total global vote weight $TW = 100,000,000$ units
- Chain C is one leaf among several

The attacker holds tokens Y (large stake A) and X (small stake t) and is LP of gauge G.

Actor	Weight	Role
Honest token Z	$W_h = 1,000,000$	permanently allocates to honest gauge H
Attacker token Y	$A = 10,000,000$	allocates to attacker-owned gauge G (attacker is G's LP)
Attacker helper X	$t = 100,000$	tiny; first-ever C allocation lands after Y exits

Consider the following voting sequence on the root chain (each row = one message to C, one nonce):

Nonce	Action	Root C weight	Dispatched $\text{chainRate} = R * (C \text{ weight} / TW)$
n3	Y allocates $A \rightarrow G$	$W_h + A = 11,000,000$	0.44 TOKEN/s
n6	Y reduces C to 0 (post-cooldown)	$W_h = 1,000,000$	0.04 TOKEN/s
n9	X allocates $t \rightarrow H$ (X's first C vote)	$W_h + t = 1,100,000$	0.044 TOKEN/s

Hyperlane messages do not guarantee that messages will be delivered in order.

Assume that n3 arrives first, followed by n9, and then n6.

After n3 (chain gate 3>0, token gate 3>0 for Y):

- $\text{chainWeight} = 11,000,000$, $\text{chainRate} = 0.44$, $G \text{ weight} = 10,000,000$, $H \text{ weight} = 1,000,000$
- $\boxtimes \text{gaugeWeight} = 11,000,000 = \text{chainWeight}$, so it is consistent.
- G fairly accrues $0.44 * 10M / 11M = 0.40 \text{ TOKEN/s}$

After n9, n6 is still in flight:

- chain gate 9>3, so $\text{applyChainState}(\text{chainRate}=0.044, \text{chainWeight}=1,100,000)$
- token gate for X: 9>0, so X's $t=100,000$ added to H
- G's weight $A=10,000,000$ is never touched. n6 is the only message that removes it, but it hasn't arrived
- Now, $\text{chainWeight} = 1,100,000$ but $\boxtimes \text{gaugeWeight} = G(10M) + H(1M) + t(0.1M) = 11,100,000$.
- Desync by 10,000,000 (G's stale weight).

Attacker calls $\text{settleGauge}(G)$, and G accrues:

```
chainRate * G_weight / chainWeight  
0.044 * 10,000,000 / 1,100,000 = 0.400 TOKEN/s
```

While honest H gauge accrues:

```
chainRate * H_weight / chainWeight  
0.044 * 1,100,000 / 1,100,000 = 0.044 TOKEN/s
```

leaf total = 0.444 TOKEN/s, while the chain C's TRUE root rate = 0.044 TOKEN/s, which is around 10x over-emission.

So, the excess (0.40 TOKEN/s) is unbacked credit to the attacker's gauge G.

Assume n6 lands $\boxtimes t = 1,800s$ (30 min) after n9, and the attacker settles G before n6 is processed:

- G's over-credited by $0.40 * 1,800 = 720$ TOKEN, fully unbacked.
- The attacker calls `settleGauge(G)` to commit this into G's ceiling, mints 720 ReceiptToken via G, and redeems.

Note 1: This 720 is an over-credit; the realized theft is capped by chain C's available root ceiling. The leaf-side over-credit rate is attacker-tunable, but the amount actually drained is bounded by chain C's pool of unredeemed legitimate emissions.

Note 2: The attack precondition depends on `n6` delayed relative to `n9`. Hyperlane's lack of ordering makes this possible as a delivery-order condition. However, an attacker might not have direct control, which makes this attack more challenging.

Note 3: The issue breaks the `Σ gaugeWeight == chainWeight` (on the leaf) invariant.

Impact

Out-of-order delivery can combine a newer, reduced chain total with stale gauge allocations, allowing a gauge to accrue excess emissions. Although root-side limits prevent unlimited inflation, the false credit can consume rewards owed to honest LPs and cause their later redemptions to fail.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/8dd41ea45ce5341e0811af4079dc5dfca032493c/LeafMessageOrchestrator.sol#L18>

Tool Used

Manual Review

Recommendation

Chain-level state and gauge-level weights must not drift apart under out-of-order delivery. Apply each chain-state update together with all gauge-weight changes it reflects, or reconcile message ordering before advancing emission accounting, so that the invariant holds.

Issue M-7: Swallowed `notifyGaugeRate` Failure Desyncs CL Pool Rate

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/142>

Vulnerability Detail

Assume the following setup:

- Base (root chain), Ethereum (leaf chain)
- Holder A and Holder B are `sTOKEN` holders on Base and own an NFT that carries allocation weight (aka voting power).
- Holder A's `sTOKEN` supplies 80% of the target gauge's allocation weight, and Holder B's `sTOKEN` supplies the remaining 20%
- Ethereum's `LeafVoter.chainRate` remains `10 TOKEN/s`
- The target CL gauge initially owns 100% of the Ethereum gauge weight, so it receives Ethereum's full `10 TOKEN/s` chain rate
- Alice is an LP on Ethereum and is the only in-range LP in the target gauge

At $T=1000$:

1. Holder A moves 80% allocation from the target gauge to another gauge on Ethereum, so the target gauge's share falls to 20% (or `2 TOKEN/s` from now onwards)
2. Base (root chain) dispatches a cross-chain message to Ethereum (leaf chain). On Ethereum, the `LeafMessageOrchestrator._handleVote()` get executed.
3. Then, it will settle `[900, 1000]` at the old `10 TOKEN/s` rate, credit `10 * 100 = 1,000 TOKEN` to the target gauge's ceiling and advance its settlement cursor to 1000
4. Removes Holder A's 80% contribution, leaving Holder B's 20% contribution to earn `2 TOKEN/s` after $t = 1000$
5. `CLGauge.notifyRewardRate(10e18, 900, 1000) -> CLPool.syncReward` will be executed without reverting. (Note: In later step, this will revert causing the issue)

Alice then claims the tokens by calling `CLGauge.claimEmissions(alice, alice)`. The gauge and pool state will be as follows:

```
gauge ceiling = gauge claimed = 1,000 TOKEN
gauge surplus = 0
pool.activeRate = 10 TOKEN/s
pool.lastUpdated = pool.rateValidUntil = 1000
```

At T=1100:

1. Holder B touches the target Ethereum gauge while leaving its aggregate share at 20%, so its effective rate remains 2 TOKEN/s
2. Same Base-to-Ethereum Hyperlane cross-chain messages get sent, and Ethereum's `LeafMessageOrchestrator._handleVote` again handles the incoming updates.
3. `LeafVoter.applyChainState` calls `_settleGauge` for [1000,1100], and commits: `effectiveShare = 2 TOKEN/s * 100 seconds = 200 TOKEN`, raising the gauge ceiling to 1,200 TOKEN and advancing `gaugeStates[targetGauge].lastSettlement` to 1100.
4. The `LeafVoter.notifyGaugeRate` get executed, but the bounded-gas call runs out of gas while forwarding:

```
try LeafVoter.notifyGaugeRate{gas: notifyRewardGasLimit}(  
  RateAndCheckpointData({gauge: targetGauge, rate: 2e18, from: 1000, to: 1100,  
    ↪ data: hex""})  
) { } catch { }
```

Note: The earlier ceiling and cursor writes remain committed, even though the Ethereum pool never receives [1000,1100] and still caches 10 TOKEN/s through timestamp 1000.

At T=1200:

1. Some keeper calls `LeafVoter.settleGauge(targetGauge)` on Ethereum, and `_settleIndex(1200)` and then `_settleGauge(targetGauge)` are triggered.
2. Since the gauge cursor is already 1100, it can derive only: `(rate, from, to) = (2e18, 1100, 1200)`
3. As a result, the gauge ceiling increased by 200 TOKEN to 1400 TOKEN

At this point:

Ethereum window	LeafVoter-authorized rate	Gauge-ceiling credit	Pool-applied rate	Pool credit
[1000,1100]	2 TOKEN/s	200 TOKEN	stale 10 TOKEN/s	2000 TOKEN
[1100,1200]	2 TOKEN/s	200 TOKEN	2 TOKEN/s	200 TOKEN
Total since Alice's claim		400 TOKEN		

The Ethereum LeafVoter permits only:

```
claimable = ceiling - claimed - surplus = 1,400 - 1,000 - 0 = 400 TOKEN
```

However, the pool credits Alice with 1,200 TOKEN, leaving an unbacked deficit of:

```
1,200 - 400 = (10 - 2) * 100 = 800 TOKEN
```

The issue is that when Alice calls `CLGauge.claimEmissions` on Ethereum, the gauge asks to mint 1,200 xTOKEN, but `LeafVoter.mintEmissions` computes only 400 TOKEN of headroom. So, it reverts with `CeilingExceeded`.

Alice also cannot bypass the failing claim by withdrawing because calling `CLGauge.withdraw(positionId)` will execute `CLGauge._withdraw -> _claimPositions -> LeafVoter.mintEmissions` before the pool stake is removed or the NFT is transferred, so the same revert will occur, leaving the position staked and the NFT inside the gauge

Impact

A failed notification is swallowed after the voter advances its accounting, making the missed rate window unrecoverable. A stale low rate underpays LPs, while a stale high rate can make rewards exceed the gauge ceiling, causing claims and withdrawals to revert and potentially locking CL positions indefinitely

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/ff6529e49e01c3f732b2692983a9acd6170deef2/LeafMessageOrchestrator.sol#L200>

Tool Used

Manual Review

Recommendation

Keep the ledger and the pool's rate view consistent. If the forwarded `notifyGaugeRate` fails, do not advance the gauge's settlement cursor/ceiling for that window, or record the missed window so the rate update can be replayed to the pool later. Independently, let withdraw skip or defer the emissions claim so a failing mint can never trap staked principal.

Issue M-8: Out-of-Order Chain Snapshots

Corrupt Leaf Emission Accounting

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/143>

Vulnerability Detail

To recap, the root vote dispatches a post-vote chain snapshot (t_{Act} , $chainRate$, $chainWeight$) to the affected leaf chains, and the leaf applies chain-level state under a latest-wins nonce gate with no ordering guarantee from the Hyperlane transport.

If vote nonce $N+1$ is delivered before vote nonce N , the leaf settles its emission index all the way to $t_{Act}\{N+1\}$ using the pre- N rate, installs $N+1$'s rate, and later discards nonce N entirely. The interval $[t_{Act}N, t_{Act}\{N+1\}]$ is therefore never settled on the leaf at the rate the root ceiling used for it.

```
// metadex/V3/src/bridge/LeafMessageOrchestrator.sol:165-169
if (_chainNonce > lastChainVoteNonce) {
    lastChainVoteNonce = _chainNonce;
    LEAF_VOTER.applyChainState(_voteMessage.chainRate, _voteMessage.chainWeight,
        ↪ _voteMessage.tAct);
}
```

As a result, the divergence between leaf reward accounting and root redemption-ceiling accounting is a permanent constant that no later message can repair

Consider the following scenario:

1. A leaf starts at $lastSettlement = T_0$ with stored $rate_0/weight_0$.
2. Root vote N_1 touches the leaf, settles root ceiling to $t_{Act}1$, stores $rate_1/weight_1$, and dispatches nonce 1.
3. Root vote N_2 later touches the same leaf, settles root ceiling over $[t_{Act}1, t_{Act}2]$ at $rate_1$, stores $rate_2/weight_2$, and dispatches nonce 2.
4. Normal transport reordering delivers nonce 2 before nonce 1.

5. The leaf applies nonce 2, settling $[T0, tAct2]$ at $rate0/weight0$, then stores $rate2/weight2$.
6. Nonce 1 arrives later and is skipped by the latest-wins chain nonce gate. The missing $[tAct1, tAct2]$ segment cannot be reconstructed.

For the two-message case, the root and leaf histories through $tAct2$ are:

```
root = rate0 * (tAct1 - T0) + rate1 * (tAct2 - tAct1)
leaf = rate0 * (tAct2 - T0)
error = (rate0 - rate1) * (tAct2 - tAct1)
```

After nonce 2 installs the final snapshot, both sides use the same forward rate. The nonzero historical error therefore remains a constant; later delivery or a later clean snapshot cannot repair it.

Note: This issue breaks the `leaf cumulative accrual == root accrued ceiling` (per chain) invariant.

Impact

Skipped rate intervals are settled using the wrong rate, permanently over- or under-crediting gauge rewards. Over-crediting can consume limited redemption headroom or cause later redemptions to fail, while under-crediting deprives LPs of rewards. Root-side limits prevent unlimited token inflation.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/ff6529e49e01c3f732b2692983a9acd6170deef2/LeafMessageOrchestrator.sol#L166>

Tool Used

Manual Review

Recommendation

Make out-of-order delivery harmless instead of discarding superseded snapshots. Either buffer and apply chain-state updates sequentially, with recovery for missing messages, or include enough authenticated rate, weight, and decay-boundary history to reconstruct every skipped interval. Never advance settlement across an unapplied rate interval.

Issue M-9: If a user has voted to a single paused chain, they cannot move their other voting power too.

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/144>

Summary

If a user has voted to many chains, even if just one gets paused, the user cannot move any of their votes.

Vulnerability Detail

Whenever a chain gets paused, the contracts disallow any outbound and inbound messages to that chain. Whenever a user votes on RootChain, they always have to send messages on all chains they've previously held votes on, as well as all chains that are included in the new vote. For this reason, if a user has a single paused chain, they're unable to move their voting power which might be towards a non-paused chain.

In a later version of the code, there is a partial workaround that - if the user has voting power towards chain0, it is the only one which can be moved - and it can only be done by rebalancing it to another NFT.

Impact

Users cannot change votes.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/main/metadex/V3/src/voter/Voter.sol#L240>

Tool Used

Manual Review

Recommendation

Whenever a user has voted towards a paused chain, allow for new votes as long as the votes towards the paused chain remain unchanged.

Issue L-1: Uint112 Cumulative Wraparound Corrupts High-Reserve TWAPs

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/133>

Vulnerability Detail

In the Notion documentation shared => Pools => v2 Pools TWAP section, it was observed that it attempts to implement a TWAP circular buffer ([feat: implements v2 pool twap circular buffer#251](#)).

The new implementation stores the reserve cumulatives as uint112 (so 32+112+112 packs one slot) to save storage/gas.

There are some incorrect assumptions.

1. It assumes that observations are nominally written every 60 seconds. However, it is not always true because 60 seconds is the minimum write spacing, not a maximum gap. The observations require pool activity, and some quiet pools can have a very long gap.
2. Did not take into account that there are some high-supply tokens that can have extremely large raw-unit reserves (e.g., SHIB)

Consider the following setup:

- Token 0: canonical SHIB, an 18-decimal ERC-20.
- Token 1: canonical Ethereum USDC, with 6 decimals.
- Reserve: 100 billion SHIB (~0.017% of SHIB supply)
- Matching liquidity: approximately 421,000 USDC at the current reference price.
- Approximate pool TVL: \$842,000.

The SHIB reserve in raw units is: $100,000,000,000 \text{ SHIB} \times 10^{18} = 10^{29} \text{ raw units}$

Over 24 hours:

```
true cumulative increase =  $10^{29} \times 86,400 = 8.64 \times 10^{33}$ 
```

However, an observation can hold only:

```
 $2^{112} = 5.192296858534827628530496329220096 \times 10^{33}$ 
```

Therefore, the SHIB cumulative completes one full uint112 turn after only:

```
 $2^{112} \div 10^{29} = 51,923 \text{ seconds} = 14.42 \text{ hours}$ 
```

After 24 hours, the stored observation contains only the remainder: `3.447703141465172371469503670779904e33`

A user calls `pool.observe([86400, 0])`, and it will calculate `averageReserve = (currentCumulative - pastCumulative) / 86400`

Assuming the stored cumulative happened to be exactly zero at the start of the window (this only simplifies the numbers), it derives:

- `pastCumulative = 0` (the zero-start illustration)
- `currentCumulative =  $3.4477 \times 10^{33}$`
- `( $3.4477e33 - 0$ ) / 86,400 =  $3.99040 \times 10^{28}$  raw units (39.904 billion SHIB)`

```
reported average  39.904 billion SHIB  
correct average   = 100 billion SHIB
```

The reserve is understated by approximately 60.1%, or about 2.5X.

Impact

Long observation gaps can cause cumulative reserves to exceed 2^{112} , losing full wraps and materially understating average reserves. This produces incorrect TWAP pricing, potentially causing financial losses for integrations relying on it for pricing, valuation, or risk decisions.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/869e1d5cfad6c6584c166ecc8175739c96c818a2/src/libraries/PoolOracle.sol>

Tool Used

Manual Review

Recommendation

Use full-width cumulative values or track wraps explicitly so the complete cumulative delta remains recoverable.

Issue L-2: Obsolete Emergency Council Getter Blocks All Per-Gauge CL Cap Updates

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/134>

Vulnerability Detail

The PR [feat: leaf voter configuration#319](#), which was merged into V3 last week. It deleted the emergencyCouncil state/getter from the metadex LeafVoter, replaced it with EMERGENCY_COUNCIL_ROLE on its AccessControl registry, and repointed the metadex GaugeFactory to hasRole(...) in V3/src/factories/GaugeFactory.sol.

However, in-scope slipstream [CLGaugeFactory](#) was not repointed as of today

```
// metadex-slipstream/contracts/gauge/CLGaugeFactory.sol:188-192
function setEmissionCap(address _gauge, uint128 _cap) external override {
    require(isGauge[_gauge], "IG");

    address emergencyCouncil = ILeafVoter(leafVoter).emergencyCouncil(); // :191
    ↪ - UNCONDITIONAL
    if (msg.sender == emergencyCouncil) { ... }
    if (hasRole(CAP_ADMIN_ROLE, msg.sender)) { ... }
    if (hasRole(CAP_OPERATOR_ROLE, msg.sender)) { ... }
```

The slipstream-side vendored contracts/gauge/interfaces/ILeafVoter.sol:45 still declares the getter, so both repos compile cleanly, but at runtime the metadex V3 LeafVoter no longer has that selector and has no fallback function, so the staticcall reverts. Thus, this will need to be fixed later.

Impact

The obsolete getter call causes every per-gauge CL emission-cap update to revert, blocking admins, operators, and the emergency council. This also prevents individual gauges from being zero-capped during emergencies, potentially allowing unwanted emissions to continue. Global default-cap updates remain functional but cannot isolate a specific gauge.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/c057a1c9642878b19a3c7216f8ce0827d1322b6d/GaugeFactory.sol#L144>

Tool Used

Manual Review

Recommendation

Align the CL factory with the current role-based LeafVoter interface.

Issue L-3: Transfers Do Not Invalidate Leaf Operator Authority

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/138>

Vulnerability Detail

It was observed that the existing leaf operator still remains after the sTOKEN NFT transfer. Consider the following example:

1. Alice owns sTOKEN `T` and appoints Mallory as its operator on chain `C`.
2. The message is delivered and the leaf stores `operator[T] = Mallory`.
3. Alice transfers `T` to Bob. The operator and its nonce remain unchanged.
4. After `T` earns 100 USDC in fees, Mallory calls `claimFees(T, Mallory, max)`.
5. Mallory receives the 100 USDC and the claim cursor advances, preventing Bob from reclaiming that interval.

Another variation of the similar issue also exists for the on-flight "grant". On a remote leaf, a pre-sale grant can similarly arrive after the transfer because the ownership change creates no newer operator nonce or generation. Consider the following example:

1. Alice owns sTOKEN `T` on the root chain and calls `setOperator(T, Mallory)` for a remote chain. The call is valid at that moment.
2. The root dispatches the message through Hyperlane, but delivery is asynchronous.
3. Before the message reaches the leaf, Alice transfers `T` to Bob.
4. The transfer does not notify the leaf or increment any ownership-version counter.
5. The original message later reaches the leaf. Since its nonce is newer than `lastOperatorNonce[T]`, the leaf accepts it and sets Mallory as operator even though Alice no longer owns `T`.

Usually, for an ERC-721 NFT, token approval will be cleared on transfer, while an old owner's `setApprovalForAll` no longer applies because they no longer own the token.

Impact

An operator appointed by the previous owner can remain authorized after an sTOKEN transfer, while delayed pre-transfer grants can also take effect afterward. The stale operator can redirect accrued fees and advance the claim cursor, permanently depriving the new owner. They may also retain local gauge-allocation control.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/7960f2c89fc37cf5ede5216d8d1d9c7138e46020/Voter.sol#L528>

Tool Used

Manual Review

Recommendation

When an sTOKEN is transferred, automatically revoke its leaf operators and reject any delayed operator assignment sent by the previous owner.

Issue L-4: Users can partially bypass voting cooldowns.

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/145>

Summary

Users can partially bypass voting cooldowns.

Vulnerability Detail

As long as the users have voting power on chain0, they can reallocate it using `withdrawToNFT/depositToNFT`. This can be used in situations where the user has voting cooldown and is usually going to miss out voting this week - this way they can move the voting power to a new NFT and earn rewards.

Impact

Circumventing intended limitation.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/main/metadata/V3/src/voter/Voter.sol#L325>

Tool Used

Manual Review

Recommendation

If it is not intentional, check the voting cooldown of the token on `rebalanceChain0`.

Issue L-5: A request from a Paused chain can be processed on RootChain, but cannot be requested on LeafChain.

Source: <https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/issues/146>

Summary

A request from a Paused chain can be processed on RootChain, but cannot be requested on LeafChain.

Vulnerability Detail

Currently, the LeafChain Voter does not allow users to request redeems when the Chain is paused. However, the RootChain does not have such restriction.

Impact

Inconsistency between the code on LeafChain and RootChain.

Code Snippet

<https://github.com/sherlock-audit/2026-06-dromos-june-24th-2026/blob/main/metadata/V3/src/voter/Voter.sol#L394>

Tool Used

Manual Review

Recommendation

Add a requirement check on RootChain that the request is not from a paused chain.

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.